Shravani Anil Patil
D20A-40

Experiment :11
Blockchain Lab

Title: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:
Hyperledger Fabric is an open-source enterprise-grade blockchain platform designed for building distributed applications. Unlike public blockchains, it is a **permissioned network**, meaning only authorized participants can access and perform transactions.

The architecture of Hyperledger Fabric consists of key components:

- **Peers** – Maintain ledger and run chaincode
- **Orderers** – Ensure transaction ordering
- **Certificate Authority (CA)** – Provides identities for participants
- **Channels** – Private communication layers for transactions

Chaincode (smart contracts) defines the business logic that governs how assets are created, updated, and transferred on the blockchain.

To run Fabric, tools like:

- Docker (for containerization)
- Git (for repository management)
- Node.js (for chaincode execution)
- Windows Subsystem for Linux (for Linux environment on Windows) are required.

This experiment demonstrates how to:

1. Set up a Fabric test network
2. Deploy chaincode
3. Perform asset lifecycle operations

Prerequisites

- Windows Subsystem for Linux (WSL)
- Ubuntu
- Docker
- Git
- Node.js & npm

Procedure:

Step 1: Update the System

Command:
sudo apt update
sudo apt upgrade -y

```
root@INFT506-34:~# sudo apt update
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:2 https://deb.nodesource.com/node_18.x nodistro InRelease [12.1 kB]
Hit:3 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:4 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Get:5 http://archive.ubuntu.com/ubuntu jammy-backports InRelease [127 kB]
Get:6 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3364 kB]
Get:7 https://deb.nodesource.com/node_18.x nodistro/main amd64 Packages [12.0 kB]
Ign:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Get:1 http://security.ubuntu.com/ubuntu jammy-security InRelease [129 kB]
Get:8 http://security.ubuntu.com/ubuntu jammy-security/main amd64 Packages [3068 kB]
Get:9 http://archive.ubuntu.com/ubuntu jammy-updates/main Translation-en [512 kB]
Get:10 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 c-n-f Metadata [19.6 kB]
Get:11 http://archive.ubuntu.com/ubuntu jammy-updates/restricted amd64 Packages [5608 kB]
Get:12 http://security.ubuntu.com/ubuntu jammy-security/main Translation-en [438 kB]
Get:13 http://security.ubuntu.com/ubuntu jammy-security/main amd64 c-n-f Metadata [14.1 kB]
Get:14 http://security.ubuntu.com/ubuntu jammy-security/restricted amd64 Packages [5356 kB]
Get:15 http://archive.ubuntu.com/ubuntu jammy-updates/restricted Translation-en [1075 kB]
Get:16 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 Packages [1262 kB]
Get:17 http://security.ubuntu.com/ubuntu jammy-security/restricted Translation-en [1025 kB]
Get:18 http://archive.ubuntu.com/ubuntu jammy-updates/universe Translation-en [316 kB]
Get:19 http://archive.ubuntu.com/ubuntu jammy-updates/universe amd64 c-n-f Metadata [30.5 kB]
Get:20 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 Packages [1025 kB]
Get:21 http://security.ubuntu.com/ubuntu jammy-security/universe Translation-en [226 kB]
Get:22 http://security.ubuntu.com/ubuntu jammy-security/universe amd64 c-n-f Metadata [22.8 kB]
Fetched 23.7 MB in 1min 57s (203 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
47 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@INFT506-34:~#
```

```
root@INFT506-34:~# sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@INFT506-34:~#
```

Explanation:
This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:
sudo apt install jq curl git nodejs npm -y
node -v
npm -v

git --version
docker --version
docker ps

```
root@INFT506-34:~# node -v
v18.20.8
root@INFT506-34:~# npm -v
10.8.2
root@INFT506-34:~# git --version
git version 2.34.1
root@INFT506-34:~# docker --version
Docker version 29.1.5, build 0e6fee6
root@INFT506-34:~# docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED        STATUS         PORTS
 NAMES
5d0000f6b4e3   mariadb:10.6                   "docker-entrypoint.s…"   6 weeks ago    Up 3 minutes   3306/tcp
 snipeit_db
3d67c106ff3d   ghcr.io/zammad/zammad:6.5.2-67   "/docker-entrypoint.…"   2 months ago   Up 3 minutes   0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
 zammad-zammad-nginx-1
c2acd1eae6e0   ghcr.io/zammad/zammad:6.5.2-67   "/docker-entrypoint.…"   2 months ago   Up 3 minutes
 zammad-zammad-scheduler-1
fd57f58245b0   ghcr.io/zammad/zammad:6.5.2-67   "/docker-entrypoint.…"   2 months ago   Up 3 minutes
 zammad-zammad-railsserver-1
64d9da24d3e4   ghcr.io/zammad/zammad:6.5.2-67   "/docker-entrypoint.…"   2 months ago   Up 3 minutes
 zammad-zammad-backup-1
5f2a8d964ea5   ghcr.io/zammad/zammad:6.5.2-67   "/docker-entrypoint.…"   2 months ago   Up 3 minutes
 zammad-zammad-websocket-1
09407446a971   postgres:17.7-alpine           "docker-entrypoint.s…"   2 months ago   Up 3 minutes   5432/tcp
 zammad-zammad-postgresql-1
e91a72c5e3ee   redis:7.4.7-alpine             "docker-entrypoint.s…"   2 months ago   Up 3 minutes   6379/tcp
 zammad-zammad-redis-1
77c366a563e3   memcached:1.6.40-alpine        "docker-entrypoint.s…"   2 months ago   Up 3 minutes   11211/tcp
 zammad-zammad-memcached-1
```

Explanation:
This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while docker ps ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples

```
root@INFT506-34:~# git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (221/221), done.
remote: Compressing objects: 100% (126/126), done.
remote: Total 15618 (delta 149), reused 95 (delta 95), pack-reused 15397 (from 3)
Receiving objects: 100% (15618/15618), 24.09 MiB | 369.00 KiB/s, done.
Resolving deltas: 100% (8801/8801), done.
root@INFT506-34:~#
```

```
root@INFT506-34:~# cd fabric-samples
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fa^C
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /root/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:--  0:00:02 --:--:--     0
 16  120M   16 20.0M    0     0   946k      0  0:02:10  0:00:21  0:01:49 1121k
```

Explanation:
This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:
curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

```
root@INFT506-34:~# git clone https://github.com/hyperledger/fabric-samples
fatal: destination path 'fabric-samples' already exists and is not an empty directory.
root@INFT506-34:~# cd fabric-samples
root@INFT506-34:~/fabric-samples# curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /root/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
```

```
Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
===> Pulling fabric Images
====>  ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====>  ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====>  ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15
====>  ghcr.io/hyperledger/fabric-baseos:2.5.15
2.5.15: Pulling from hyperledger/fabric-baseos
Digest: sha256:654bd4554bf97c70902e56d530294795372518da7ae2a7cdfccbe397a878c513
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
```

Explanation:
This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:
docker images | grep fabric

```
root@INFT506-34:~/fabric-samples# docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
root@INFT506-34:~/fabric-samples#
```

Explanation:
This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and order are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version

```
root@INFT506-34:~/fabric-samples# export PATH=$PATH:$HOME/fabric-samples/bin
root@INFT506-34:~/fabric-samples# export FABRIC_CFG_PATH=$HOME/fabric-samples/config
root@INFT506-34:~/fabric-samples# peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger

root@INFT506-34:~/fabric-samples#
```

Explanation:
This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running the peer version verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:
cd ~/fabric-samples/test-network
./network.sh up createChannel

```
root@INFT506-34:~/fabric-samples# cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveldb with crypto from 'crypto
gen'
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/root/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
WARN[0000] No services to build
[+] up 7/7
 ✔Network fabric_test                 Created                                    0.0s
 ✔Volume compose_orderer.example.com  Created                                    0.0s
 ✔Volume compose_peer0.org1.example.com Created                                  0.0s
 ✔Volume compose_peer0.org2.example.com Created                                  0.0s
```

```
ed with status (VALID) at localhost:9051
2026-04-07 11:43:58.528 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [08e465dad2f26121c0b15c68e167f2ebda1bf2c32295ae2423e9bb7c58931ce2] committ
ed with status (VALID) at localhost:7051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

Explanation:
This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```
./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript
```

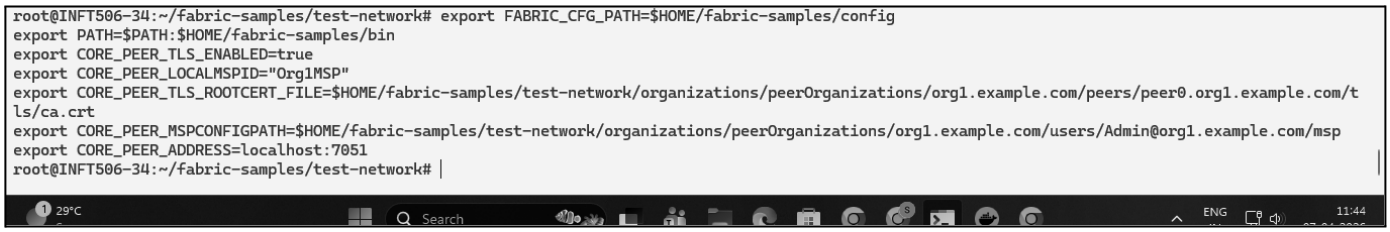
```
root@INF1506-34:~/fabric-samples/test-network# ./network.sh deployCC  -ccn basic  -ccp ../asset-transfer-basic/chaincode-javascript  -ccl javascr
ipt
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...
Using organization 1
+ peer lifecycle chaincode queryinstalled --output json
+ jq -r 'try (.installed_chaincodes[].package_id)'
+ grep '^basic_1.0:1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629$'
+ test 1 -ne 0
+ peer lifecycle chaincode install basic.tar.gz
```

Explanation:
This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051



Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \

--tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function":"InitLedger","Args":[]}'

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.
com-cert.pem \
-C mychannel \
-n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
--waitForEvent \
-c '{"function":"InitLedger","Args":[]}'
2026-04-07 11:52:33.549 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [cbe2e1a8048ef95418c228b192dee1d7ff4a14947549ab0a8166865435ecc369] committ
ed with status (VALID) at localhost:7051
2026-04-07 11:52:33.550 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [cbe2e1a8048ef95418c228b192dee1d7ff4a14947549ab0a8166865435ecc369] committ
ed with status (VALID) at localhost:9051
2026-04-07 11:52:33.550 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
root@INFT506-34:~/fabric-samples/test-network#
```

Explanation:
This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets

Command:
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","
Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"Appra
isedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner"
:"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
root@INFT506-34:~/fabric-samples/test-network#
```

Explanation:
This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset
Command:
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
root@INFT506-34:~/fabric-samples/test-network#
```

Explanation:
This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","yellow","10","Alice","500"]}'
```

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exampl
e.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"CreateAsset","Args":["asset10","yellow","10","Alice","500"]}'
2026-04-07 12:00:21.104 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [9b065c629e96af164059bcb494038aa21e9ee4b40281d06d913b1c4bdb414d22] committ
ed with status (VALID) at localhost:9051
2026-04-07 12:00:21.105 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [9b065c629e96af164059bcb494038aa21e9ee4b40281d06d913b1c4bdb414d22] committ
ed with status (VALID) at localhost:7051
2026-04-07 12:00:21.105 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"{\"ID\":\"
asset10\",\"Color\":\"yellow\",\"Size\":10,\"Owner\":\"Alice\",\"AppraisedValue\":500}"
root@INFT506-34:~/fabric-samples/test-network#
```

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exampl
e.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"InitLedger","Args":[]}'
2026-04-07 11:59:20.742 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [e3d456a0f6ee4c6132a8e71ea451491f49cb1f2926215aefc7691e0cfa0c3401] committ
ed with status (VALID) at localhost:9051
2026-04-07 11:59:20.743 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [e3d456a0f6ee4c6132a8e71ea451491f49cb1f2926215aefc7691e0cfa0c3401] committ
ed with status (VALID) at localhost:7051
2026-04-07 11:59:20.743 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
root@INFT506-34:~/fabric-samples/test-network# peer chaincode query \
  -C mychannel \
  -n basic \
  -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"AppraisedValue":400,"Color":"red","ID":"asset2","
Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"Appra
isedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner"
:"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner,

and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

```
peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Bob"]}'
```

```
root@INFT506-34:~/fabric-samples/test-network# peer chaincode invoke \
  -o localhost:7050 \
  --ordererTLSHostnameOverride orderer.example.com \
  --tls \
  --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exampl
e.com-cert.pem \
  -C mychannel \
  -n basic \
  --peerAddresses localhost:7051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
  --peerAddresses localhost:9051 \
  --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
  --waitForEvent \
  -c '{"function":"TransferAsset","Args":["asset10","Bob"]}'
2026-04-07 12:01:59.386 +0530 0001 INFO [chaincodeCmd] ClientWait -> txid [eee32197d9cd297abdd9c5800d4a0a48934210e17f8ea648fc3298aa45db7ab4] committ
ed with status (VALID) at localhost:9051
2026-04-07 12:01:59.387 +0530 0002 INFO [chaincodeCmd] ClientWait -> txid [eee32197d9cd297abdd9c5800d4a0a48934210e17f8ea648fc3298aa45db7ab4] committ
ed with status (VALID) at localhost:7051
2026-04-07 12:01:59.387 +0530 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Alice"
root@INFT506-34:~/fabric-samples/test-network#
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

./network.sh down

```
root@INFT506-34:~/fabric-samples/test-network# ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✔Container orderer.example.com         Removed                                                                    0.8s
 ✔Container peer0.org2.example.com      Removed                                                                    1.4s
 ✔Container peer0.org1.example.com      Removed                                                                    1.5s
 ✔Network fabric_test                   Removed                                                                    0.4s
 ✔Volume compose_peer0.org1.example.com Removed                                                                    0.0s
 ✔Volume compose_orderer.example.com    Removed                                                                    0.0s
 ✔Volume compose_peer0.org2.example.com Removed                                                                    0.0s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908
e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:e396fa43457a17b304bb3b26314fb3cb58ba10f8bc73879e81f3cde20298e7e5
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf
0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:429fe488758bb9495f7cd4969cc5336c04932468f2eaaa3bd0bf2b98cde109ff
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
aecba0016ef2: Download complete
Digest: sha256:1487d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
Status: Downloaded newer image for busybox:latest
root@INFT506-34:~/fabric-samples/test-network#
```

Conclusion :

Successfully implemented a Hyperledger Fabric network, deployed chaincode, and performed asset creation, querying, and transfer operations.